CSC1185 Neural Language Modelling

Week 6: Transformers

Jurafsky & Martin, Feb 2024 3rd edition draft, Sections 10.1-10.6

Overview

Transformers

1. Introduction
2. Transformers are Self-attention Networks
3. Architecture Overview
4. Self-attention
5. Parallelising Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

Overview

Transformers

1. Introduction
2. Transformers are Self-attention Networks
3. Architecture Overview
4. Self-attention
5. Parallelising Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

Introduction

- In Weeks 6 and 7 we focus on the concepts of **pretraining** and **finetuning**, i.e. carrying out a round of more expensive general training of a language model, which is then either finetuned in a far cheaper second stage or used in conjunction with other separately trained components.
- Pretraining is usually done as **autoregressive training**, and a huge amount of knowledge can be learnt in this way simply through learning language from vast amounts of text.
- The resulting **large language models (LLMs)** exhibit remarkable performance on all sorts of natural language tasks because of the knowledge they learn in pretraining.
- LLMs have been transformative for tasks where we need to produce text, like summarisation, machine translation, question answering, or chatbots. However, many knowledge-driven tasks soon followed. The now dominant LLM architecture is the **transformer.**
- Transformers use the central concept of **self-attention** which can be thought of as a way to build contextual representations of a word's meaning that integrate information from surrounding words, helping the model learn how words relate to each other over large spans of text.
- Language modelling is based on a simple idea: self-training to iteratively predict words left-to-right from earlier words.
- It turns out that this is not just a powerful way to learn language, but also to acquire all sorts of real-world information, factual or otherwise, grounded in the physical world or not.
- Underlying the power of language models is the fact that almost any task can be construed (modelled) as word prediction, if we think about it in the right way.

Overview

Transformers

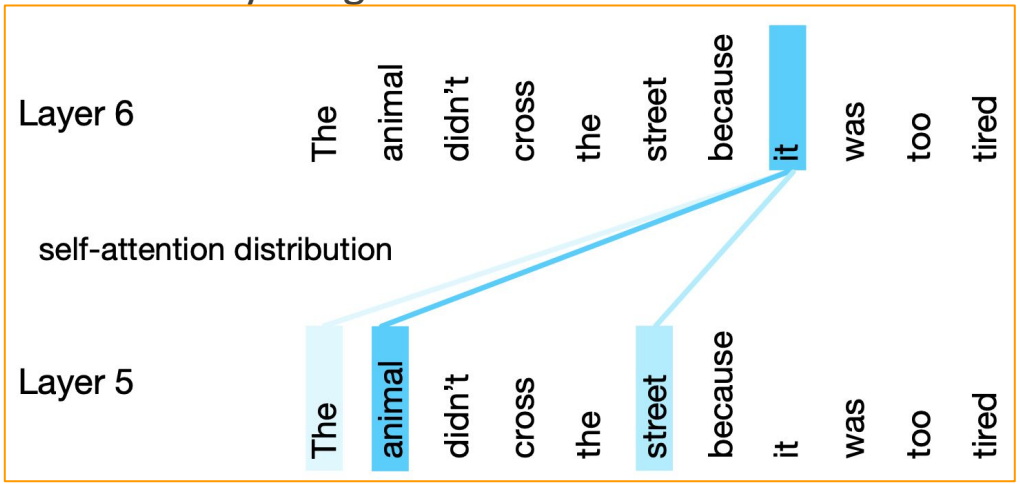
1. Introduction
2. Transformers are Self-attention Networks
3. Architecture Overview
4. Self-attention
5. Parallelising Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

Transformers are self-attention networks

- Transformers are composed of a series of layers, and the idea is that from layer to layer, we build up richer and richer **contextualised** representations of the meanings of input words (more precisely, tokens).
- At each layer, to compute the contextualised representation of word w_i we combine information from the representation of word w_i at the previous layer with information from the representations of the neighbouring words.
- These contextualized representations are created for each word at each position.
- Static word vectors like those computed by word2vec compute the same representation for every word regardless of where it is encountered which creates problems e.g. with synonymy and antonymy.
- In contrast, transformers learn to produce **contextualised word vectors**, at inference time computed on the fly to represent each word in the specific current context in which it is encountered.
- The mechanism by which transformers compute these contextualised representations is **self-attention** which looks at the context and tells us how to integrate representations from words in the context from layer $k-1$ to build representations for words in layer k.

Transformers are self-attention networks

- Recall computation of attention weight distributions from Week 5.
- Self-attention also creates weight distributions α as part of the computation of word representations at each layer.
- In the example below (simplified from a real transformer, Uszkoreit, 2017), the self-attention distribution for the word ‘it’ at layer 6 gives the highest weight to (pays most **attention** to) ‘animal’ and secondary weights to ‘The’ and ‘street’.
- Note that the transformer attending highly to ‘animal’ is sensible, since ‘it’ corefers with ‘animal’, and to get the pronoun right we need to know the antecedent.
- In terms of self-attention, the representation for ‘it’ draws on the representation for ‘animal’.



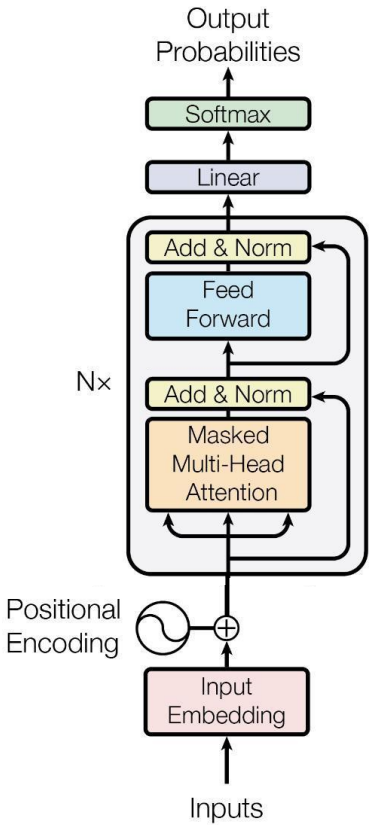
Overview

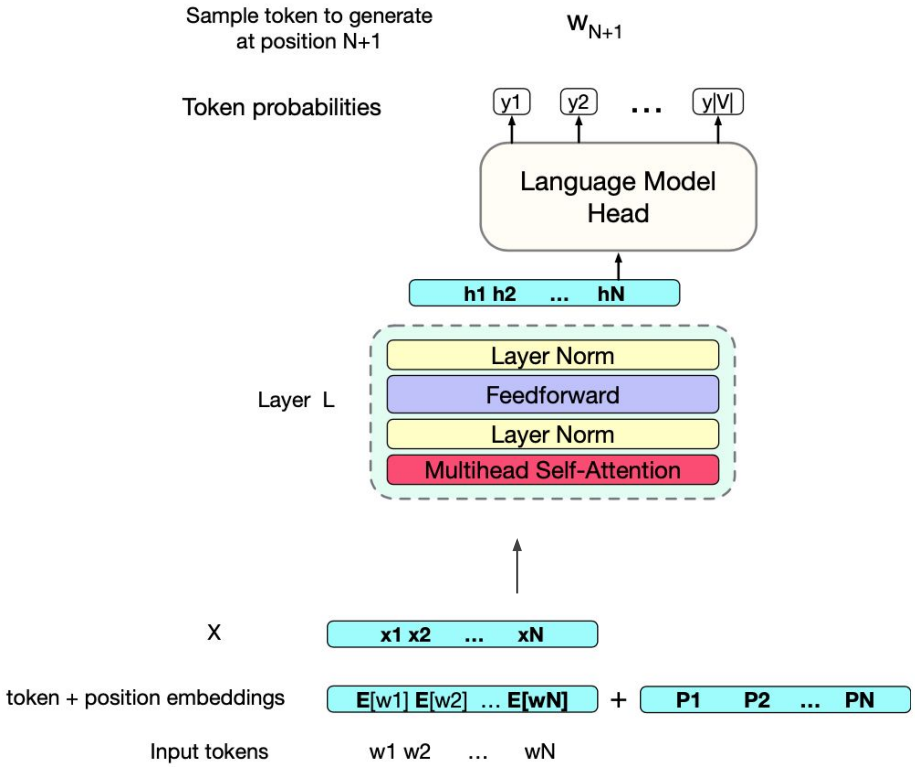
Transformers

1. Introduction
2. Transformers are Self-attention Networks
3. Architecture Overview
4. Self-attention
5. Parallelising Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

The whole decoder-only transformer architecture

- The whole decoder-only transformer architecture at a glance: far right, from Jurafsky & Martin.
- Near right, the decoder-only version of the original diagram from Vaswani et al. (2017).



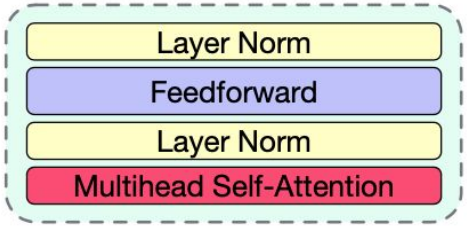


The whole decoder-only transformer architecture

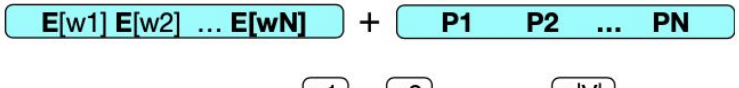
We will look at the transformer components in this order:

- (single-head) self-attention
- multihead self-attention

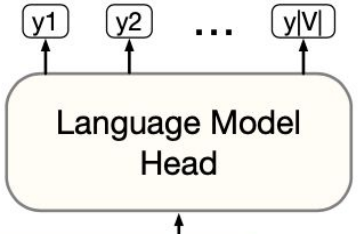
- the transformer block (layer) as a whole, and its remaining components



- token and position embeddings



- the language model head



Overview

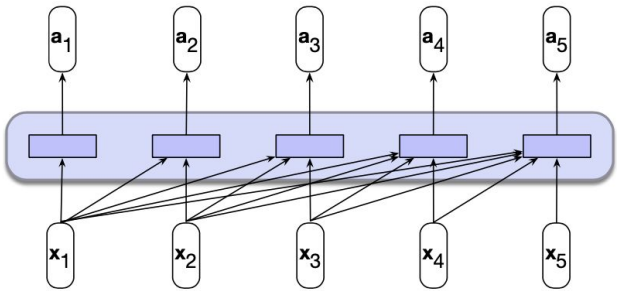
Transformers

1. Introduction
2. Transformers are Self-attention Networks
3. Architecture Overview
4. Self-attention
5. Parallelising Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

Self-attention

Multihead Self-Attention

- Note, this week we're looking only at **causal attention**, or backward looking attention.
- A self-attention layer maps input sequences $\mathbf{x}_1,\ldots,\mathbf{x}_n$ to same-length output sequences $\mathbf{a}_1,\ldots,\mathbf{a}_n$.
- When processing each item in the input, the model has access to all and only the inputs up to and including the current one, but not inputs beyond the current one.
- The computation performed for each input item is independent of the computations for all the other items.
- Causal attention ensures that we can use this approach to create language models and use them for autoregressive generation.
- Independence of computations means both forward inference and training can be parallelised.
- The core intuition of attention is that it is a way of comparing the current input item to other words (tokens) within a given sequence in a way that reveals their relevance in the current context.
- The result of these comparisons is then used to compute an output sequence for the current input sequence. E.g. the computation of $\mathbf{a}_3$ is based on a set of comparisons between the input $\mathbf{x}_3$ and its preceding elements $\mathbf{x}_1$ and $\mathbf{x}_2$, and to $\mathbf{x}_3$ itself.



Self-attention

- Recall from Week 5 using dot product to measure relevance in terms of similarity in the context of LSTM encoder-decoders.
- Softmax was used to turn a vector of dot products into a vector of weights α (the attention mask), where higher weights indicated higher relevance.
- To compute output $\mathbf{a}_i$ we then computed the weighted sum of input embeddings:

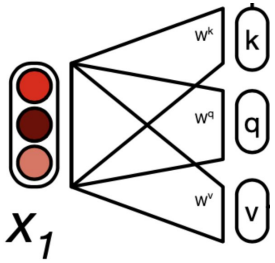
$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j \tag{10.7}$$

- The crucial difference in transformer self-attention is that we first obtain specialised representations intended to reflect the three different roles in which an input embedding is used in making predictions:
  - As the current focus of attention when being compared to all of the other previous inputs. Here it functions as a **query**.
  - As a previous input being compared to the current focus of attention. Here it functions as a **key**.
  - As a **value** (the input value) used to compute the output for the current focus of attention.

Self-attention

- To capture these three different roles, transformers introduce weight matrices W^Q, W^K, and W^V to project each input vector $\mathbf{x}_i$ into a representation of it as a query, key and value, respectively.

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}}; \;\; \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{K}}; \;\; \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^{\mathbf{V}} \qquad (10.8)$$

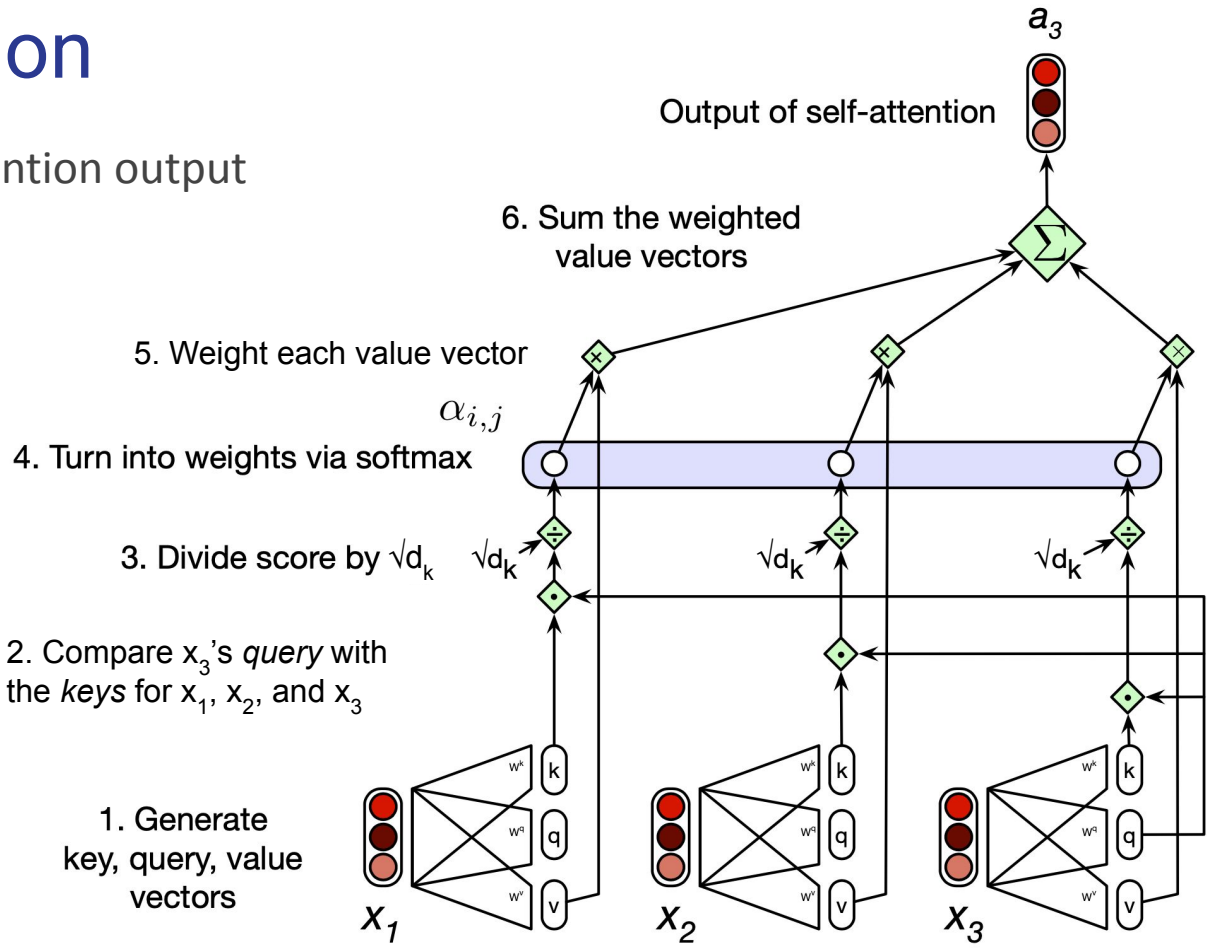


- In computing a given $\mathbf{a}_t$, only the current query vector $\mathbf{q}_t$ is used, not the other query vectors computed for other input values in the sequence
- However, all keys $\mathbf{k}_i$ and values $\mathbf{v}_i$ computed for inputs in the sequence so far are used, each in a way suited to their role.
- Query vector $\mathbf{q}_i$ is combined via dot product with each key vector computed from each input, **scaled** by the length of **k** and put through softmax to give self-attention mask $\alpha_{i,j}$ which is then used to weight $\mathbf{v}_j$ in computing the current output token $\mathbf{a}_i$ *:*

$$\mathbf{a}_i = \sum_{j \le i} \alpha_{ij} \mathbf{v}_j \qquad (10.10)$$

Self-attention

Computing self-attention output element a_3:



Self-attention

- To make the various computations work, the vectors and matrices as usual have to have the correct dimensions.
- Inputs **x**, outputs **y**, and all intermediate vectors (e.g. attention output vector **a**), all have the same dimensionality $1 \times d$, d being the overall dimensionality of the transformer.
- Key and query vectors are of dimension d_k, and value vectors of dimension d_v which can be different from each other and from d.
- In the original transformer work (Vaswani et al., 2017), d was 512, d_k and d_v were both 64.
- The shapes of the transform matrices are then:

$$\mathbf{W}^Q \in \mathbb{R}^{d \times d_k}, \mathbf{W}^K \in \mathbb{R}^{d \times d_k}, \text{ and } \mathbf{W}^V \in \mathbb{R}^{d \times d_v}$$

Overview

Transformers

1. Introduction
2. Transformers are Self-attention Networks
3. Architecture Overview
4. Self-attention
5. Parallelising Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

Parallelising self-attention

- We have so far looked at self-attention in terms of computing a single 'current' attention layer output $\mathbf{a}_i$ when presented with a 'current' input token $\mathbf{x}_i$.
- However, since the computations of attention layer output elements are independent of each other, we can once again put all inputs in an input matrix **X** which is of dimensionality $N \times d$ where N is the number of input embeddings and d the input sequence length (and the overall dimensionality of the transformer).
- Multiplying **X** (N×d) by the key, query, and value weights (of shape $d \times d_k$, $d \times d_k$, $d \times d_v$) produces matrices $\mathbf{Q} \in \mathbb{R}^{N \times d_k}$, $\mathbf{K} \in \mathbb{R}^{N \times d_k}$, and $\mathbf{V} \in \mathbb{R}^{N \times d_v}$ containing the key, query, and value vectors:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{\mathbf{Q}}; \;\; \mathbf{K} = \mathbf{X}\mathbf{W}^{\mathbf{K}}; \;\; \mathbf{V} = \mathbf{X}\mathbf{W}^{\mathbf{V}} \qquad (10.15)$$

- The query-key comparisons can then also be computed simultaneously by multiplying **Q** and $\mathbf{K}^T$ in a single matrix multiplication (result being of shape $N \times N$).
- We can also then scale these score matrices, take the softmax, and then multiply the result by **V**, in single matrix multiplications.
- The final result is matrix **A** of shape $N \times d$ containing the output from the entire self-attention step for the entire sequence of N tokens.

Parallelising self-attention

- Computing **A** has then been reduced to the following computation:

$$\mathbf{A} = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \;=\; \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\mathbf{V} \qquad (10.16)$$

- Need to fix just one thing: the calculation in QK^T results in a score for each query value to every key value, including those that follow the query.
- This is inappropriate in the setting of language modeling: guessing the next word is pretty simple if you already know it!
- To fix this, the elements on the corresponding side of the matrix diagonal are set to $-\infty$), thus eliminating any knowledge of words that follow in the sequence.

N

q1•k1	−∞	−∞	−∞	−∞
q2•k1	q2•k2	−∞	−∞	−∞
q3•k1	q3•k2	q3•k3	−∞	−∞
q4•k1	q4•k2	q4•k3	q4•k4	−∞
q5•k1	q5•k2	q5•k3	q5•k4	q5•k5

N

Overview

Transformers

1. Introduction
2. The Intuition behind Transformers
3. Architecture Overview
4. Self-attention
5. Parallelised Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

Multi-head attention

- The graphs and computations we've seen so far were for a single **attention head**.
- Transformers actually compute self-attention with multiple attention heads that reside in parallel layers at the same depth in a model, each with its own set of parameters.
- By using these distinct sets of parameters, the idea is that each head can learn different aspects of the relationships among inputs, at the same level of abstraction.
- The heads each compute their key, value, and query embeddings separately, with the rest of the self-attention computation remaining unchanged.
- The model dimension is still d, the input and output, the key and query embeddings still have dimensionality d_k, and the value embeddings dimensionality d_v .
- The output of a multi-head layer with h heads consists of h matrices of shape $N \times d_v$. To make use of these matrices in further processing, they are concatenated to produce a single output with dimensionality $N \times hd_v$.
- To reshape the output back down to the original output dimension for each token, multiplying the concatenated $N \times hd_v$ matrix output by $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d}$ yields the self-attention output **A** of shape $[N \times d]$, suitable to be passed through residual connections and layer norm.

Multi-head attention

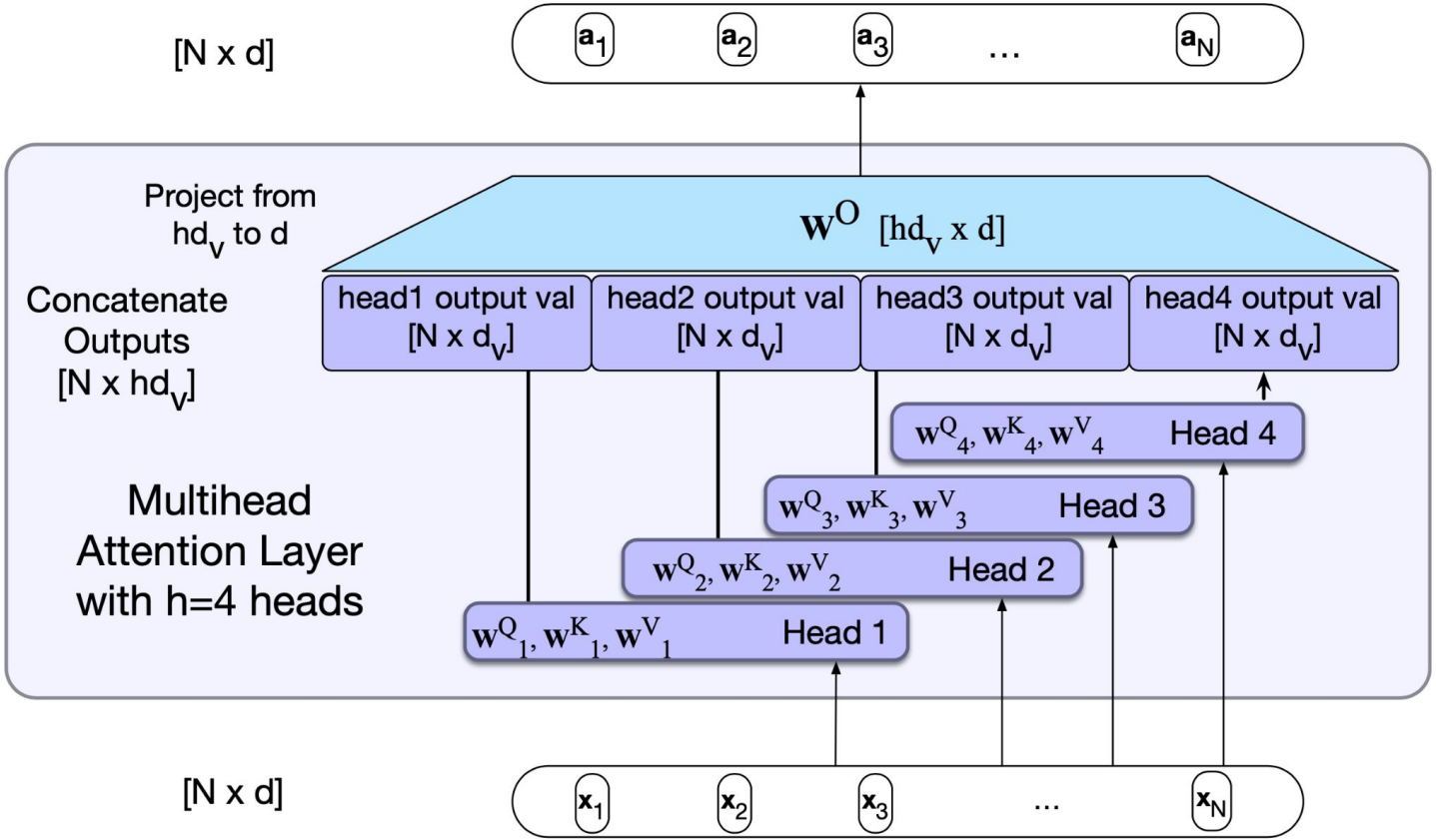
- We then have the following final equations to describe the computations in a multi-head attention layer:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_i^Q \; ; \; \mathbf{K} = \mathbf{X}\mathbf{W}_i^K \; ; \; \mathbf{V} = \mathbf{X}\mathbf{W}_i^V \tag{10.17}$$

$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \tag{10.18}$$

$$\mathbf{A} = \text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \ldots \oplus \mathbf{head}_h)\mathbf{W}^O \tag{10.19}$$

Multi-head attention

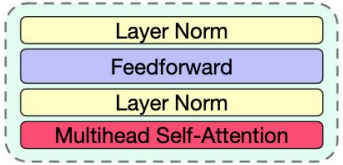


Overview

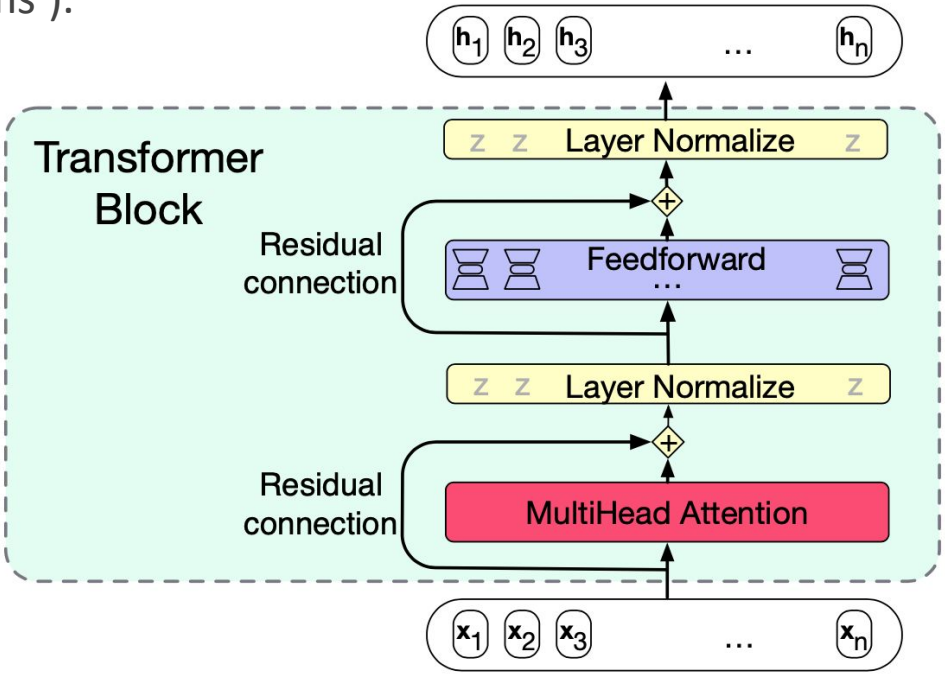
Transformers

1. Introduction
2. The Intuition behind Transformers
3. Architecture Overview
4. Self-attention
5. Parallelising Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

The Transformer block



- Transformer block has a multihead self-attention layer, a feedforward layer, residual connections, and two normalising layers (often called ‘layer norms’).
- The **feedforward layer** contains *N* parallel, fully-connected 2-layer FFNs, one at each position. The FFN weights are the same for each position in one layer, but different from one layer to another.
- The FFNs are independent and can be computed in parallel.
- It is common to make the dimensionality of the hidden layer of the FFNs much larger than the model dimensionality.
- E.g. in the original transformer, the latter was 512, the former 2,048.)



The Transformer Block: Residual connections

- **Residual connections** are connections that pass representations on unchanged from an earlier layer to a later layer.
- Allowing information from the activation going forward, and therefore the gradient going backwards, to skip a layer improves learning and gives later layers direct access to information from lower layers (He et al., 2016).
- Transformers have residual connections within transformer blocks as well as between.
- Transformer residual connections are implemented simply by (elementwise) adding a layer's input vector to its output vector before passing it forward.
- The transformer block shown in the last slide has residual connections around both the attention and feedforward sublayers.

The Transformer Block: Layer norm

- The summed vectors are normalised using **layer normalisation** (Ba et al., 2016) which is computed as follows:

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma} \tag{10.22}$$

$$\text{LayerNorm} = \gamma \hat{\mathbf{x}} + \beta \tag{10.23}$$

- 10.22 subtracts the mean μ from each vector component and divides it by the standard deviation σ.
- The result of this computation is a new vector with zero mean and a standard deviation of one.
- $\hat{\mathbf{x}}$ can be thought of as a variation of the standard score, or z-score, applied to a vector
- γ and β are two learnable parameters, representing gain and offset values.
- Layer normalisation (usually called layer norm) is one of many forms of normalisation that can be used to improve training performance in deep neural networks by keeping the values of a hidden layer in a range that facilitates gradient-based training.

The Transformer Block: Putting it all together

- The function computed by a transformer block can be expressed as:

$$\mathbf{O} = \text{LayerNorm}(\mathbf{X} + \text{SelfAttention}(\mathbf{X})) \tag{10.24}$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{O} + \text{FFN}(\mathbf{O})) \tag{10.25}$$

- Or we can break it down into component computations yielding $\mathbf{T}^i$ (of shape N×d):

$$\mathbf{T}^1 = \text{SelfAttention}\ (\mathbf{X}) \tag{10.26}$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1 \tag{10.27}$$

$$\mathbf{T}^3 = \text{LayerNorm}\ (\mathbf{T}^2) \tag{10.28}$$

$$\mathbf{T}^4 = \text{FFN}\ (\mathbf{T}^3) \tag{10.29}$$

$$\mathbf{T}^5 = \mathbf{T}^3 + \mathbf{T}^4 \tag{10.30}$$

$$\mathbf{H} = \text{LayerNorm}\ (\mathbf{T}^5) \tag{10.31}$$

- Recall that input and output dimensions of transformer blocks are matched, so they can be stacked.
- Transformers for LLMs stack many blocks, from 12 (T5, GPT-3-small) to 96 (GPT-3 large), to even more for more recent models.

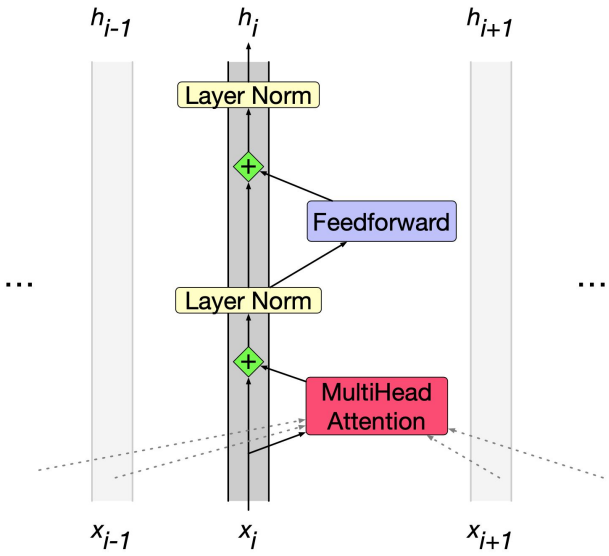
Overview

Transformers

1. Introduction
2. The Intuition behind Transformers
3. Architecture Overview
4. Self-attention
5. Parallelising Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual Stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

The residual stream

- The **residual stream** view of transformers looks at things from the perspective of an individual input token vector $\mathbf{x}_i$ of dimensionality d as it is processed through each transformer block.
- Most transformer components take such a single token vector as input, and produce an output vector also of dimensionality d.
- In FFN layers, over the N tokens in a batch, we simply use the identical feedforward layer weights and biases $\mathbf{W}_1$, $\mathbf{W}_2$, $\mathbf{b}_1$ and $\mathbf{b}_2$ for each ith token.
- Similarly, the layer norm function takes a single d-dimensional vector and produces a normalised d-dimensional version of it as output.
- Outputs of feedforward and attention layers get added into the stream, with layers constantly copying information up from earlier embeddings.



- Only the multi-head attention component reads information from the other residual streams in the context in addition to the current token stream.
- We can view attention heads as literally moving attention from the residual stream of a neighbouring token into the current stream.

The residual stream

- We can think of the various transformer components as adding new views of the representation back into the constant residual stream.
- Reflecting this perspective of progressively manipulating the *d*-dimensional input vector, we can reformulate our transformer equations as follows:

$$\mathbf{t}_i^1 = \text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \cdots, \mathbf{x}_N]) \tag{10.32}$$

$$\mathbf{t}_i^2 = \mathbf{t}_i^1 + \mathbf{x}_i \tag{10.33}$$

$$\mathbf{t}_i^3 = \text{LayerNorm}(\mathbf{t}_i^2) \tag{10.34}$$

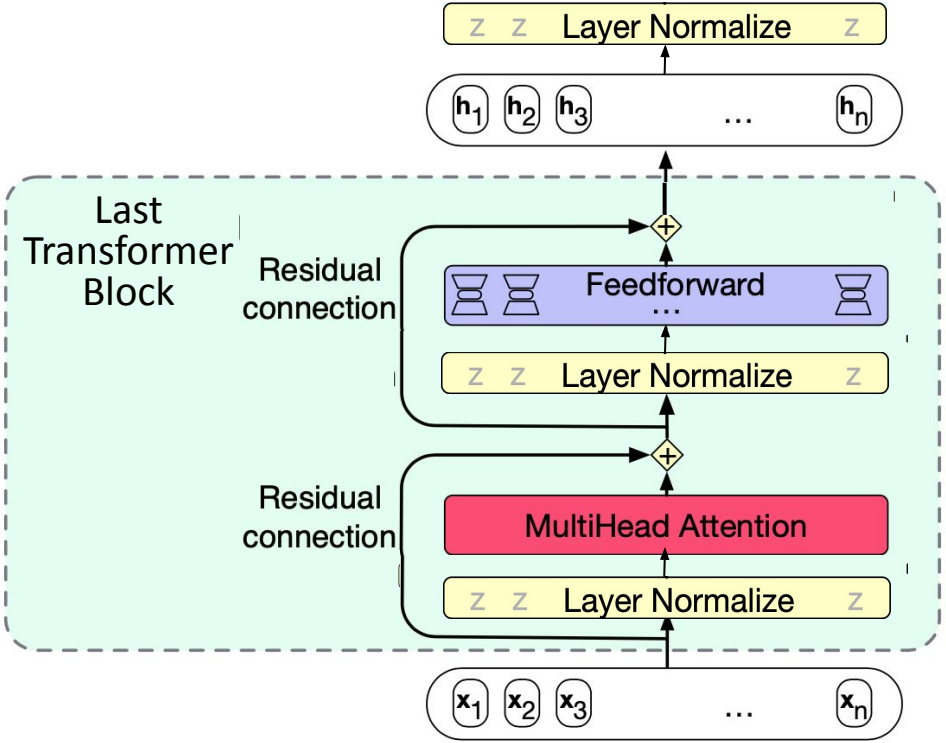
$$\mathbf{t}_i^4 = \text{FFN}(\mathbf{t}_i^3)) \tag{10.35}$$

$$\mathbf{t}_i^5 = \mathbf{t}_i^4 + \mathbf{t}_i^3 \tag{10.36}$$

$$\mathbf{h}_i = \text{LayerNorm}(\mathbf{t}_i^5) \tag{10.37}$$

Pre-norm vs. Post-norm architecture

- **Pre-norm** is an alternative form of the transformer architecture that is commonly used because it performs better in many cases.
- Here, the layer norm happens before the layers rather than after them, so before the attention layer and before the feedforward layer instead of afterwards.
- After the last transformer block, there is a single extra layer norm, just below the language model head layer that we will look at next.



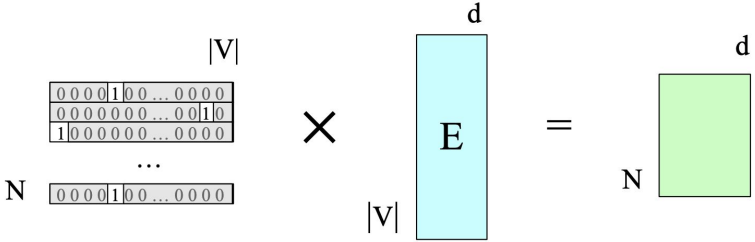
Overview

Transformers

1. Introduction
2. The Intuition behind Transformers
3. Architecture Overview
4. Self-attention
5. Parallelising Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

Token and position embeddings

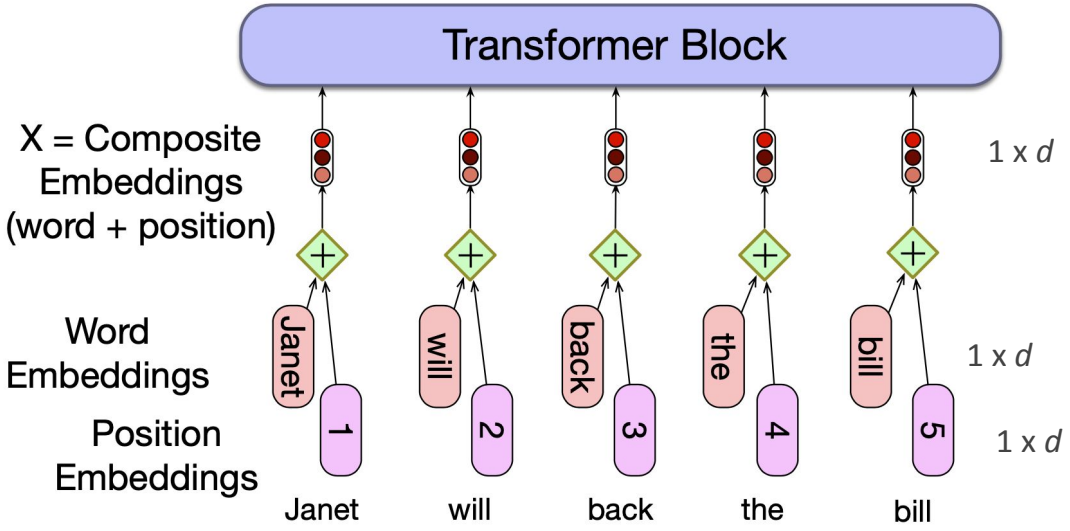
- Just as we saw in previous weeks, input tokens are converted to their embedding forms by one-hot vector look up in the embedding matrix:



- The resulting embeddings have information about token meaning, but are devoid of information about where in the token sequence a token is located, i.e. they hold no positional information.
- Transformers explicitly add information about the position of tokens in the sequences, as that is an important part of the meaning of a token in a given context.
- This is done by explicitly combining token embeddings with **positional embeddings** specific to each position in the input sequence.

Token and position embeddings

- There are different approaches to creating positional embeddings.
- The simplest method, called **absolute position**, is to start with randomly initialised embeddings corresponding to each possible input position up to some maximum length.
- For example, just as we have an embedding for the word ‘back’, we have an embedding for the position 3.
- Positional embeddings are learnt along with other parameters during training (like token embeddings).



Token and position embeddings

- The absolute position embeddings approach will have plenty of training examples for the initial input positions and increasingly fewer towards the length limit.
- Embeddings for tokens in positions nearer the limit may be poorly trained and not generalise well to unseen inputs.
- An alternative approach is to use a static function that maps integer inputs to real-valued vectors in a way that captures the inherent relationships among the positions. That is, it captures the fact that position 4 in an input is more closely related to position 5 than it is to position 17.
- E.g. a combination of sine and cosine functions with different frequencies was used in the original transformer work.
- More complex positional embedding methods exist, e.g. representing relative instead of absolute position, often implemented in the attention mechanism at every layer instead of once at the initial input.

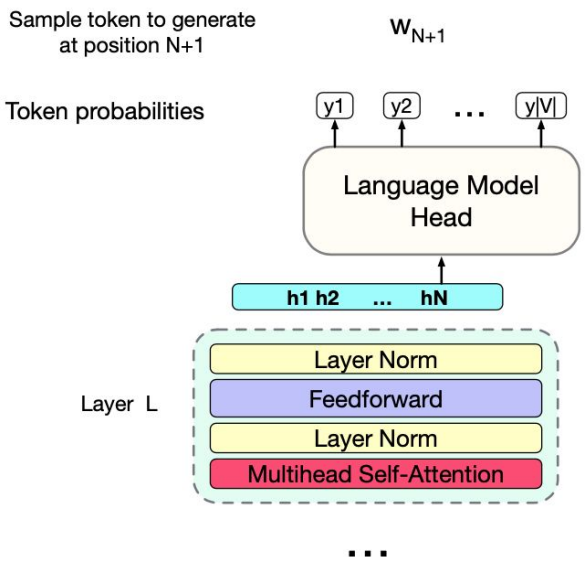
Overview

Transformers

1. Introduction
2. The Intuition behind Transformers
3. Architecture Overview
4. Self-attention
5. Parallelising Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

The language modelling head

- In general, the term **head** is used to mean the additional neural circuitry that is added on top of the basic neural architecture to enable the model to perform a specific task, e.g. language modelling or classification.
- The **language modelling head** in a transformer is the final feed-forward layer plus softmax that assigns probabilities to output tokens.
- Recall that language models assign a probability to every possible next word conditioned on a context, giving us a distribution over the entire vocabulary.



- For transformer language models, the context is the size of the transformer's context window, which can be quite large: 2048, 4096 tokens or more for large models.

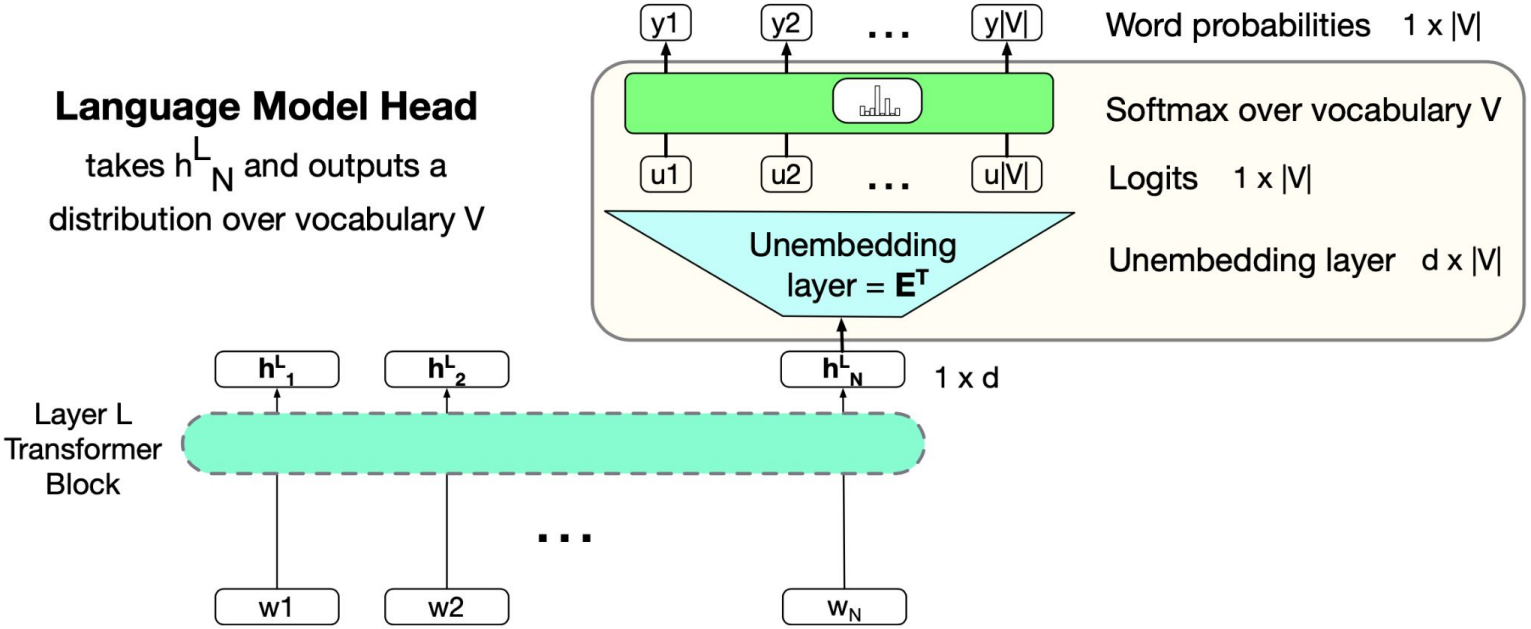
The language modelling head

- The job of the language modeling head is to take the output of the last token at the last layer (output embedding of shape 1×d) and produce a probability distribution over vocabulary items.
- The linear layer projects from the output $\mathbf{h}^L_N$ which represents the output token embedding at position N from the final block L (shape $1 \times d$) to the logit vector $\mathbf{u}$ (dimensionality $1 \times |V|$) that will have a single score for each of the $|V|$ vocabulary words.
- This linear layer can be learned separately, but more commonly the matrix is tied to (the transpose of) the embedding matrix $\mathbf{E}$.
- Thus at the input stage of the transformer the embedding matrix (of shape $|V| \times d$) is used in the usual way to map from a one-hot vector over the vocabulary (of shape $1 \times |V|$) to an embedding (of shape $1 \times d$).
- And then in the language model head, the transpose of the embedding matrix $\mathbf{E}^T$ (of shape $d \times |V|$) is used to map back from an embedding (shape $1 \times d$) to a vector over the vocabulary (shape $1 \times |V|$).
- In the learning process, $\mathbf{E}$ will be optimised to be good at doing both mappings.
- We therefore sometimes call the transpose $\mathbf{E}^T$ the **unembedding layer** because it performs this reverse mapping.
- Finally, a softmax layer turns the logit vector to a probability distribution over vocabulary items.

The language modelling head

Language Model Head

takes h^L_N and outputs a distribution over vocabulary V



Overview

Transformers

1. Introduction
2. The Intuition behind Transformers
3. Architecture Overview
4. Self-attention
5. Parallelising Self-attention
6. Multi-head Attention
7. Transformer Blocks
8. Residual stream, Pre-norm vs. Post-norm
9. Token and Position Embeddings
10. The Language Modelling Head
11. Summary and References

Summary

- **Transformers** are the standard architecture for building large language models.
- The original transformer work (Vaswani et al., 2017) built on previous research in **self-attention** (Week 5) and **memory networks**.
- Self-attention can be thought of a way to build **contextual representations** of a word's meaning that integrate information from surrounding words, helping the model learn how words relate to each other over large spans of text.
- The idea of using a soft weighting over the encodings of input words to inform a generative decoder was first developed by Graves (2013) for handwriting generation, and Bahdanau et al. (2015) for MT.
- This idea was extended to self-attention by dropping the need for separate encoding and decoding sequences and instead seeing attention as a way of weighting the tokens in collecting information passed from lower layers to higher layers (Ling et al., 2015; Cheng et al., 2016; Liu et al., 2016).

Summary

- Other aspects of the transformer, including the terminology of **key**, **query**, and **value**, came from memory networks.
- **Memory networks** add an external read-write memory to networks, by using an embedding of a query to match keys representing content in an associative memory (Sukhbaatar et al., 2015; Weston et al., 2015; Graves et al., 2014).
- A standard application of transformers is to language modelling, in a setting often called **causal language modelling** or **autoregressive language modelling**, in which we iteratively predict words left-to-right from earlier words.
- These language models, like the feedforward and RNN language models we saw in Weeks 2, 4 and 5, are thus **self-trained**: no external supervision is needed, the model simply learns to guess the next word in a sequence given the sequence so far.
- Given a large enough corpus of text, large language models exhibit remarkable performance on all sorts of natural language tasks because of the knowledge they learn in this way.

References

- Jurafsky & Martin (2023) Speech and Language Processing, Feb 2024 3rd edition draft, Sections 10.1-10.6.

Further Reading:

- Uszkoreit, J. 2017. Transformer: A novel neural network architecture for language understanding. Google Research blog post, Thursday August 31, 2017.
- He, K., X. Zhang, S. Ren, and J. Sun. 2016. Deep residual learning for image recognition. CVPR.
- Kenton, J. D. M. W. C., & Toutanova, L. K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of NAACL-HLT (pp. 4171-4186).
- Toan Q. Nguyen and Julian Salazar (2019) Transformers without Tears: Improving the Normalization of Self-Attention. In Proceedings of the 16th International Conference on Spoken Language Translation, Hong Kong. Association for Computational Linguistics.
- Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L. & Liu, T.. (2020). On Layer Normalization in the Transformer Architecture. Proceedings of the 37th International Conference on Machine Learning, in Proceedings of Machine Learning Research 119:10524-10533.

Questions?